Find the Function

A code search challenge: match plain-English descriptions to Python functions

Everything you need for the day is in here: the task, the data, how we score, and the rules.

TL;DR

Task	For each English query, rank the 1000 Python functions in its pool so that the one it describes comes first
Data	250k training pairs, 5000 dev queries (with answers), 5000 test queries (answers hidden)
Score	MRR@100 on the test set. Higher is better, 1.0 is perfect
Compute	Your laptop or Google Colab (free GPU allowed)
AI tools	Allowed, agents too. We'll show you Junie and can help with it on the day
Teams	3 people

1 What's the task?

You've definitely been here: you know there's a function somewhere in the codebase that *parses the config* or *downloads a file*, but you don't know what it's called. Searching for names only works if you already know them. **Semantic code search** means finding code from a description of what it does.

The difficulty is that natural language and code use different vocabularies. A query says *"download videos by URL"*; the matching function may contain `get_content`, `match1` and `download_urls`, and never mention the word video. Your system has to connect the two.

Your goal: for each query, put the correct function as high as you can in your ranking. Any approach is fine as long as it follows the rules: better keyword search, classic ML, neural networks, pretrained models, or a mix. **Where to start:** Open notebook.ipynb. It walks you through the data, gives you a working pipeline, and helps you inspect what your system gets wrong. Use evaluate.py to check your predictions.

2 Data

2.1 Files

Everything is in the `data/` folder of the challenge's repo.

File	Rows	What's inside
`train.jsonl.gz`	250653	matching pairs: `id`, `query`, `code`
`dev_queries.jsonl`	5000	`query_id`, `pool`, `query`
`dev_candidates.jsonl`	5000	`candidate_id`, `pool`, `code`
`dev_answers.csv`	5000	the correct `candidate_id` for each dev query
`test_queries.jsonl`	5000	same as dev; this is what you're ranked on
`test_candidates.jsonl`	5000	same as dev

2.2 Queries, candidates and pools

- A **query** is the docstring someone originally wrote for a function. Usually it's a sentence or two, but some are long and include parameter descriptions.
- A **candidate** is a Python function with its docstring and comments removed.
- Dev and test are each split into 5 **pools** of 1000 queries and 1000 candidates. Every query has **exactly one** correct candidate, always in its **own pool**. So you only ever rank 1000 functions per query.
- IDs are random, so don't bother looking for patterns in them.

Example

Query: *Downloads Dailymotion videos by URL.*

Correct candidate (shortened):

```
def Func(arg_0, arg_1='.', arg_2=True, arg_3=False, **arg_4):
    arg_5 = get_content(rebuilt_url(arg_0))
    arg_6 = json.loads(match1(arg_5, r'qualities":({.+?}),"'))
    ...
    download_urls([arg_9], arg_7, arg_11, arg_12, arg_1=arg_1, arg_2=arg_2)
```

2.3 Important differences between train and dev/test

- In **dev and test**, every function is called `Func`, and arguments and variables are renamed `arg_0`, `arg_1`, ... Every occurrence of the function's original name became `Func`, so `plt.show(...)` can turn into `plt.Func(...)`. Library calls, attribute names and strings are untouched.
- In **train**, functions keep their real names which will not help you at test time.
- Train, dev and test come from different GitHub repositories, and functions that were copies or near-copies across them have been removed.

More details and quirks are in `docs/data_card.md`.

3 Submission format

One CSV file with a header and three columns:

```
query_id,candidate_id,rank
Tq00000,Tc00763,1
Tq00000,Tc00823,2
...
```

- List **up to 100** candidates per test query, best first, ranked $1, 2, \ldots$ No repeated ranks or candidates within a query.
- Candidates have to come from the query's own pool.
- A query that's missing from the file just scores 0. A broken file gets rejected, so run `evaluate.py` on a dev file first to check the format.

4 Evaluation

4.1 MRR@100

For each query q, let r_q be the rank at which your file lists the correct candidate. The query's score is:

$$\mathrm{RR}_q = \begin{cases} 1/r_q & \text{if } r_q \le 100, \\ 0 & \text{otherwise (or if } q \text{ is missing),} \end{cases} \qquad \mathrm{MRR@100} = \frac{1}{|Q|} \sum_{q \in Q} \mathrm{RR}_q$$

In words: rank 1 gets you a full point, rank 2 half a point, rank 10 a tenth, and anything below your top 100 gets nothing. Say the correct answers for four queries land at ranks $1, 2, 10$ and "not found". Then MRR $= (1 + 0.5 + 0.1 + 0)/4 = 0.4$. Random guessing gets about 0.005.

`evaluate.py` also prints **Recall@1** (the share of queries you got exactly right) and **Recall@10** (the share with the answer in your top 10) to help you understand your system.

4.2 Final ranking

Each final submission is validated for format and scored on the hidden test answers, using the same `evaluate.py` you have. Highest MRR@100 wins. Ties: higher Recall@1 first, then whoever submitted earlier. There's no live leaderboard. Use dev to track your progress.

4.3 Special award

We will also recognize the team that demonstrates the best technical oversight and critical decision-making when using AI agents during this challenge. To be considered, include a short reflection in *METHOD.md* file.

5 Rules

The short version: build whatever you like, but don't look up the answers or use the original data beyond what we give you. Also make sure it's possible to rerun your predictions.

5.1 AI tools and agents

Coding assistants and agents are welcome. We'll demo **Junie** at the beginning, and can help you with it on the day. Other tools are fine too, but you're on your own with them. There are just two important boundaries:

- Your retrieval system must run independently. The prediction step cannot call hosted AI services such as ChatGPT, Claude, or Gemini APIs. Use code and models you can run yourselves.
- **What your agent does is on you.** If it downloads the original dataset, that's your team breaking the rules. Check what it's doing.

6 What to submit

1. **Predictions**: `submission_test.csv` (format in Section 3).
2. **Code**: a link to repo with everything you need to train and predict, plus a `requirements.txt` and a few lines on how to run it.
3. **METHOD.md:** A short record of your experiments.
4. **Model files**, if you have any: in the repo, or a Google Drive/Hugging Face link that works without logging in. At most 2 GB.